

Lesson 0007 — Depth and geometry filtering

~85–110 MIN

Depth evidence

ANCHORED TO: `GEOMETRY_FILTER.PY` · `EVALUATION/M4C_GEOMETRY_FILTER/REPORT.MD`

Depth is a second check for a proposed detection. Learn the actual statistics, thresholds, decision branches, and limits of the M4c1 geometry filter.

HOW TO USE THIS LESSON

Read the explanation, inspect the visual, make a prediction, and then compare it with the repository evidence. The goal is a model of this project that you can explain—not a list of framework slogans.

PREREQUISITES AND SCOPE

Start after image coordinates in Lesson 0001, candidate filtering in Lesson 0002, and RGB/depth synchronization in Lesson 0006. This lesson introduces only the camera geometry needed to audit the implemented filter.

Literal code behavior $\neq$ validated height geometry

1 · Z-DEPTH TRUTH

floor · 479 mm

I

top · 465 mm

Closer surface → smaller depth
for z-depth along the optical axis.

2 · CURRENT PREDICATE

depth > floor + 30

WRONG SIGN

It selects farther pixels, so the
variable name "raised" is misleading.

3 · EVIDENCE BOUNDARY

Observed KEEP/REJECT counts
describe current software behavior.

They do not validate physical
height classification.

Correct + regression-test first.

POLICY NOTE

Sparse per-box depth → KEEP favors recall/availability; it is not automatically a safety guarantee.

Corrected overview — established z-depth geometry, observed code behavior, and policy choices are separated explicitly. The former portrait sheet is superseded; the source SVG remains available.

Retrieval warm-up

Answer from memory before revealing the cues. Difficulty here is useful: it strengthens retrieval and shows what deserves review.

1. Trace one synchronized RGB/depth pair to the three published outputs.

► Reveal recall cues

2. How does an invalid whole depth frame differ from sparse depth in one box?

► Reveal recall cues

The question this lesson answers

How can depth provide a second check for a cube-like object without replacing YOLO? The current geometry filter treats a decoded RGB detection as a hypothesis, estimates a local reference depth, selects pixels using the implemented numeric predicate, measures that subset's shape, and returns `KEEP` or `REJECT` with a reason.

BY THE END, YOU SHOULD BE ABLE TO

- distinguish optical-axis depth from pixel coordinates and full Euclidean range;
- derive the sign of a “closer than floor” test with concrete numbers;
- trace clipping, valid-depth selection, annulus estimation, subset selection, extent, ratio, and decision order;
- explain why depth spread is not a mathematical proof of planarity;
- separate observed filter outcomes from invalid physical interpretations caused by the known sign defect.

MENTAL MODEL

YOLO proposes; depth checks the proposal. If YOLO proposes nothing, the geometry filter has no box to inspect. If per-box depth is too sparse, the current code favors recall/availability by keeping the candidate with a low-quality reason. That is a policy choice, not automatically a safety guarantee.

WHAT A DEPTH IMAGE CONTAINS

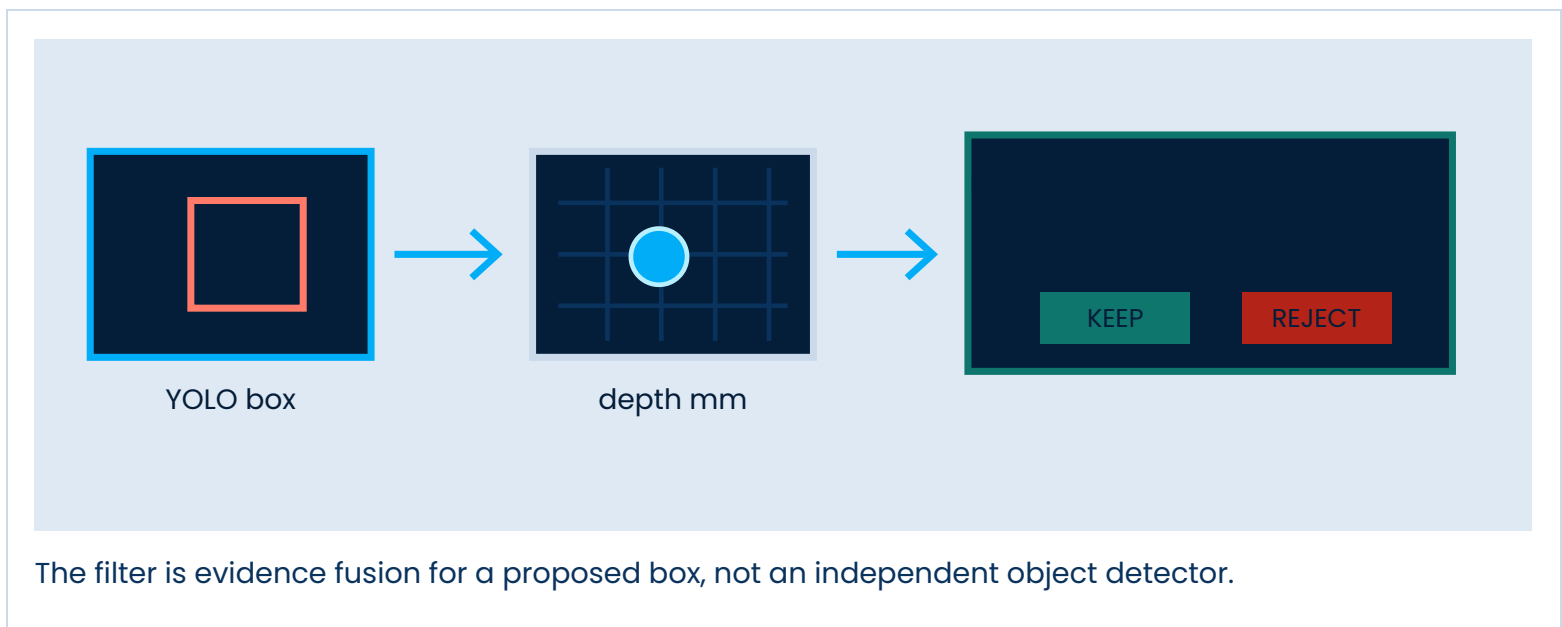
A depth image is a grid aligned to camera rays. Each valid pixel stores distance along the camera's optical z -axis, not color brightness and not automatically the full Euclidean distance to the camera center. In this project the expected storage is `uint16` millimetres. Smaller valid z means closer to the camera; zero means invalid or unknown under the ROS depth convention.

Image coordinates conventionally use u or x horizontally to the right and v or y vertically downward from the top-left. Those pixel indices are not millimetres. Camera intrinsics and a valid depth value are needed before a pixel can be related to metric 3-D coordinates.

1. Start with a candidate, not with a color mask

`compute_geometry()` receives a depth image, integer box coordinates, and runtime parameters. It does not scan the entire frame for cubes. This preserves the project's YOLOv5-first approach and

makes the geometry branch a post-filter for candidate boxes.



2. Coordinate alignment is an assumption worth checking

The filter indexes `depth_mm[y, x]` using the RGB detection box coordinates. M4cl treated the depth stream as color-registered for this purpose. If RGB and depth are not aligned, a correct 2D box can sample the wrong physical surface. The current code does not perform a separate registration transform inside `geometry_filter.py`; the camera topic contract must therefore be verified on hardware.

Registration means mapping measurements from the depth camera geometry into the RGB image geometry, using calibration and the relative camera pose. Same array dimensions, synchronized timestamps, and plausible-looking overlays are useful checks but not independent proof of pixel-perfect registration.

3. Units and invalid values

The helper expects a two-dimensional `uint16` array in millimetres. It ignores zeros and values at or above `max_depth_mm=4000`. The node's callback checks the type and shape before calling the helper. A unit mistake—metres treated as millimetres, or RGB values treated as depth—would make every threshold meaningless.

Invalid depth is not the same as very far depth. Zero may result from occlusion, reflective/absorptive surfaces, sensor range limits, or registration holes. Discarding invalid

samples avoids treating them as geometry, but it reduces evidence and can activate the filter's fail-open policy.

4. Estimate the local floor with an annulus

For a box, the helper trims an inner box by `inset_px=1` and builds an outer ring with `annulus_outer_px=15`. It removes the box area from that ring and takes the median of valid annulus pixels as `floor_mm`. The median is intended to be robust to a few invalid pixels or small texture changes, but the M4c report warns that a tilted-down camera can make the annulus measure floor slope rather than true object height.

An **annulus** here simply means a rectangular ring around the candidate. The median is the middle sorted value; unlike the mean, a few extremely near or far pixels cannot drag it as strongly. It remains only a local-reference estimate: another object in the ring, missing depth, perspective, or misregistration can make it wrong.

The box and annulus are clipped to image boundaries. Near an image edge, the ring becomes asymmetric and may contain fewer samples. The inner `inset_px` also discards the candidate border to reduce mixed object/floor pixels, but a tight YOLO box can then remove part of the already-small object evidence.

Interactive: step through one box

- 1 · Use the surrounding annulus, not the object interior, to estimate a local reference depth.

5. The primary test currently has a reversed physical sign

For every valid in-box depth value, the helper selects values numerically greater than `floor_mm + raised_mm`. With `raised_mm=30`, at least `min_raised_frac=0.20` must satisfy that literal predicate or `decide()` reports `flat`. Call this the *selected fraction* when reasoning about current behavior; the variable name `raised_frac` overstates what it measures.

CONFIRMED DEFECT, NOT AN UNRESOLVED CONVENTION

ROS depth images encode distance along the optical z-axis. A cube top closer to the camera has a smaller depth than the surrounding floor, as the project's own example also shows (465 mm top versus 479 mm floor). Therefore $\text{depth} > \text{floor_mm} + 30$ selects points farther from the camera, not a raised top. Until production code is corrected and regression-tested, KEEP/REJECT results must be described as observed behavior of this predicate—not validated height classification. [ROS depth convention · implemented predicate]

Concrete sign check: if surrounding floor is 500 mm and a cube top is 450 mm, the top is 50 mm closer. A physically consistent 30 mm “raised” test would select $\text{depth} < 500 - 30$, so 450 passes. The current test asks for $\text{depth} > 500 + 30$, so it rejects the top and instead selects values beyond 530 mm.

6. The selected subset gets a 3D extent

When more than five selected pixels exist, the helper reconstructs a small point set from pixel coordinates and depth. With camera intrinsics f_x and f_y , it estimates horizontal and vertical world extents, plus the observed depth range. The ratio is computed from the sorted extents as long/short. This is a rough geometric test, not a full point-cloud reconstruction, and it inherits the sign defect in the selection step.

The intrinsics f_x , f_y , c_x , and c_y describe the camera's pinhole projection: focal scale in x/y and the image location of the optical center. Pixel width alone is not a physical width—a 40-pixel object far away is larger in the world than a 40-pixel object nearby—so depth and intrinsics are needed to estimate metric extent.

The full pinhole relation is $X=(u-c_x)Z/f_x$, $Y=(v-c_y)Z/f_y$, $Z=\text{depth}$. The current helper uses the simpler extent approximation $\Delta X \approx \Delta u \cdot Z/f_x$ and $\Delta Y \approx \Delta v \cdot Z/f_y$ at one representative depth. That approximation is most credible for a compact subset with similar depths; it is not a calibrated object pose or full point cloud.

7. Aspect ratio rejects long thin selected subsets

The current default is $\text{max_ratio}=1.2$. If the selected subset's long/short ratio exceeds that value, `decide()` returns a reason beginning `aspect: .` M4cl's green-bag run produced a ratio around 1.45–1.5 and was rejected. A ratio nearer 1.0 is more square, but because selection is currently reversed, this statistic is not validated cube geometry.

The implementation sorts three extents—horizontal, vertical, and depth range—and divides largest by smallest when the smallest exceeds 1 mm. This is a rough cuboid-likeness heuristic. Perspective, partial visibility, tight boxes, noise, and the chosen subset can change it; many non-cubes can also have a ratio near 1.

8. Depth spread rejects irregular selected subsets

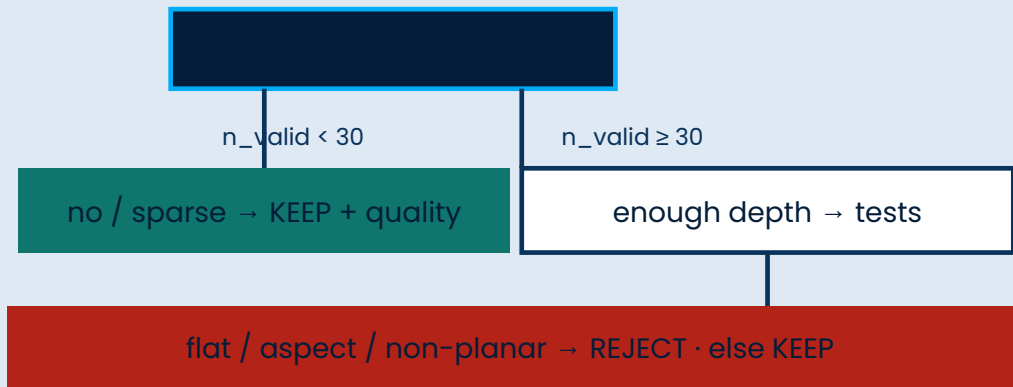
The helper measures the standard deviation of the selected subset's depth. The default `max_planar_top_stddev_mm=30` rejects a high-spread subset with a `no_planar_top: reason`. The green bag's reported spread was about 86 mm. A real planar top should have low spread after correct selection, but the current statistic is evidence about the implemented subset—not proof of an object's identity.

Calling this “planar” is shorthand. Standard deviation measures spread around one depth value; it does not fit a plane. A tilted plane can have substantial depth spread, while a curved or stepped surface may sometimes have little spread over a small patch. A true planarity test would fit a plane in 3-D and inspect residuals.

9. The decision function has an intentional fail-open branch

`decide()` first returns `KEEP` with a `no_depth_stats` reason when statistics are unavailable, and keeps a candidate with `low_depth_quality` when `n_valid < n_min` (default 30). With sufficient valid depth, it applies flat, aspect, and planar-top checks. This improves recall and availability under weak per-box depth, but may pass a false positive; whether that is safe depends on what consumes the detection.

Fail-open means uncertainty allows the candidate through; **fail-closed** would reject when depth evidence is unavailable. Neither policy is universally correct. Fail-open protects recall but can pass hazards; fail-closed protects against false positives but can hide real cubes when the sensor is weak. The downstream task determines the safer choice.



The current implementation distinguishes “not enough evidence” from “evidence says reject.”

10. What M4c1 actually demonstrated

Object/evidence set	Observed filter result	What can be said
Blue cardboard tissue box	0/30 kept	all recorded candidates removed; physical reason not validated
Blue cardboard package	0/61 kept	all recorded candidates removed; physical reason not validated
Blue decal	0/31 kept	all recorded candidates removed; physical reason not validated
Green raised Uber Eats bag	0/64 kept	all recorded candidates removed under aspect/spread decisions
Real blue cube	22/29 kept	partial preservation; below 90% target
Real red cube	0/30 kept	recorded tight top-face candidates were not preserved

THE REPORT PREDATES THE SIGN AUDIT

The M4c report sometimes calls distractor rejection “solved” or physically correct. Those rows remain valid as observed input/KEEP/REJECT counts, but their height-based interpretation is not valid while the implemented selection sign is reversed. This lesson intentionally narrows the claim instead of copying the report’s stronger wording.

11. A new failure mode prevents overclaiming

M4c1 also recorded a gray folded cloth that received `green_cube` candidates around confidence 0.40. Because the cloth was folded and had depth variation, geometry did not reject it. The report notes that the production confidence threshold of 0.50 removes this particular case. One filter can solve one failure mode and leave another; evidence should be scoped that way.

12. M5 live consequence

In the live empty run, 439 output messages contained zero kept detections; cumulative diagnostics also recorded 991 `flat` rejects and one aspect reject. This demonstrates the observed empty-scene outcome, not physically correct floor-height reasoning, because the selection sign is wrong. In the cubes-in-frame M5c2 run, the recorded candidates did not yield usable red/blue cube boxes at the operating threshold. Geometry cannot create a missing candidate, but the evidence does not isolate the model as the only possible cause.

13. Check your understanding

Why is the annulus outside the box?

1. ☐ To estimate the predicted class from neighboring depth pixels.
2. ☐ To estimate a local reference depth around the candidate.
3. ☐ To estimate TensorRT latency around the candidate region.

Why can a real cube still be rejected?

1. ☐ Its box may be tight or its sampled depth may fail checks.
2. ☐ Its class score may be replaced by the annulus median value.
3. ☐ Its coordinates may be converted directly from pixels into millimetres.

What is the key limit of geometry filtering?

1. ☐ It cannot estimate depth statistics inside an existing candidate.
2. ☐ It cannot reject candidates that violate configured geometry checks.
3. ☐ It cannot create a candidate that YOLO never proposed.

Floor depth is 500 mm and cube-top depth is 450 mm. Which direction identifies the top as at least 30 mm closer?

1. ☐ depth > 530
2. ☐ depth < 470
3. ☐ depth == 500

14. Synthetic sign exercise

Predict the result first: a 20×20 candidate contains a 450 mm top and its surrounding image is 500 mm floor. Then run the actual helper. This does not touch hardware:

```
.venv-m2/bin/python -c "import numpy as np; from recognition_of_different_colored_cubes.geometry_filter
```

Observable result: floor 500 , in-box median 450 , selected fraction 0.0 , and a flat rejection. The result exposes the reversed predicate: an idealized raised top is rejected even with perfect synthetic depth.

15. Explain it back

Explain this without reading: “What does the filter literally compute, why is its depth sign physically reversed, and why does a missing YOLO candidate stop the geometry branch?”

Use: annulus, reference median, selected fraction, 3D extent, aspect, planar top, fail-open.

What this lesson intentionally postpones

This lesson intentionally postpones camera calibration derivation, full RGB/depth registration mathematics, plane fitting, point-cloud processing, and production-code repair. It teaches the implemented filter and its measured boundary first; any sign fix requires new synthetic and recorded-data regression evidence.

Lesson 0008 owns threshold evaluation and regression design. Calibration derivations, registration algorithms, and full point-cloud processing are outside the present nine-lesson

scope; the OpenCV calibration and ROS depth references below are the next sources if implementation work requires them.

Primary references and next lesson

- ROS REP-118: Depth Images — depth units, invalid values, and the optical-axis distance convention.
- OpenCV camera-calibration model — the pinhole projection used when turning pixels and depth into metric extents.
- Project README and the linked canonical documentation.
- Curriculum plan — scope, teaching contract, and project truth.
- Lesson 0008 — Evaluation and Fine-Tuning: design the evidence loop for improving the positive-detection gap.

CONTINUE WITH THE TEACHING AGENT

Ask the agent to clarify anything uncertain, challenge your explanation with a counterexample, or assess your explain-back. A learning record should be created only after you demonstrate the idea in your own words.

← Previous lesson

Course home

Quick reference

Mission

Next lesson →